UTEC
UNIVERSIDAD DE INGENIERÍA
Y TECNOLOGÍA

P2 - Modelo predictivo de adicción al smartphone

Hiter Frank Rubio Peralta
Alumno

Jossep Jorge Froilan Choroco Neyra
Alumno

Joel David De la Cruz Cardenas
Alumno

Rodrigo Octavio Lucatoni Flores Ruiz
Alumno

Victor Andres Diaz Garcia
Alumno

Curso:
Introduccion a Ciencia de Datos e Inteligencia Artificial

21 de Octubre del 2026

CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL

+51 936 603 314
pandascore@gmail.com

Tabla de contenidos

1. Resumen **4**

2. Planteamiento del problema **4**

3. Datos y preparación **4**

3.1. Librerías 4

3.2. Carga de datos 5

3.3. Renombrado y dataset limpio 5

3.4. Decisiones de selección de variables 6

4. Selección de variables y codificación **7**

5. División en entrenamiento y prueba **8**

6. Modelos predictivos **9**

6.1. Regresión logística 9

6.2. Random forest 9

7. Evaluación **10**

7.1. Comparación contra la línea base 10

7.2. Matrices de confusión 11

7.3. Curvas ROC 12

8. Importancia de variables y autopsia de errores **12**

8.1. Importancia de predictores 13

8.2. Autopsia de errores 14

9. Conclusiones de P2 **15**

10. Limitaciones y consideraciones éticas **15**

Listado de Tablas

1. Decisiones sobre cada variable, heredadas de la P1 y aplicadas al modelo de la P2. . . 7

2. Métricas de cada modelo en el conjunto de prueba, junto con la línea base (predecir siempre 'Adicto'). 11

3. Coeficientes de la regresión logística (variables escaladas). 14

4. Características de los registros mal clasificados por el mejor modelo, comparadas con el conjunto completo. 15

Listado de Figuras

1. Matrices de confusión en el conjunto de prueba: regresión logística (izquierda) y random forest (derecha). 11

2. Curvas ROC de ambos modelos en el conjunto de prueba. 12

3. Importancia de cada predictor en el random forest (impurity-based). 13

1. Resumen

Continuando con el análisis exploratorio de la P1, en este informe entrenamos y evaluamos modelos predictivos para la etiqueta binaria de adicción (`etiqueta_adiccion`) a partir del conjunto de datos de 7,500 usuarios. Partimos de la base saneada en la P1: sin vacíos reales, sin duplicados y con `nivel_adiccion` excluida por fuga de información. Codificamos las variables categóricas, dividimos los datos en entrenamiento y prueba manteniendo el desbalance de clases (70.8 % adictos) y comparamos una regresión logística con un random forest. Como línea base, un modelo que siempre predijera "adicto" acierta el 70.8 %: ese es el piso que cualquier modelo debe superar. Los resultados se reportan con accuracy, F1, matriz de confusión y ROC-AUC Pedregosa et al. (2011), y se cierra con una autopsia de los errores del modelo.

2. Planteamiento del problema

En la P1 establecimos que la intensidad de uso es el determinante asociado a la etiqueta de adicción: más de 8 horas de pantalla diaria o más de 4 horas en redes sociales concentran el 100 % de los casos positivos. La pregunta que guía este informe es:

> ¿Qué tan bien puede un modelo de aprendizaje automático predecir la etiqueta de adicción a partir de las variables de uso, descanso y contexto personal, y qué errores comete?

La literatura reciente aplicada a este tipo de problemas usa exactamente enfoques de aprendizaje automático sobre features de uso comportamental Azeez (2026), y las librerías empleadas son las estándar del ecosistema Python Pedregosa et al. (2011).

3. Datos y preparación

3.1. Librerías

```
# ======================================================================
# CONFIGURACIÓN ÚNICA DEL REPORTE
# Librerías y estilos se ejecutan una sola vez
# ======================================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, Latex
from scipy import stats
import math

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, roc_curve, accuracy_score,
                             f1_score)
```

```
sns.set_theme(style="whitegrid")
```

Funciones de presentación heredadas de la P1: la función `tabla()` reproduce el estilo de tabla (`booktabs` con filas alternadas) directamente desde Python.

3.2. Carga de datos

A diferencia de la P1, el dataset se carga desde la carpeta local `datos/`, sin necesidad de internet. `keep_default_na=False` evita que pandas convierta la categoría "None" de `addiction_level` en valores nulos (el error que detectamos en la P1):

```
df = pd.read_csv("datos/smartphone_use.csv", keep_default_na=False)
n_filas, n_columnas = df.shape
print(f"Dimensiones: {n_filas:,} filas x {n_columnas} columnas")
print(f"Valores nulos: {int(df.isna().sum().sum())}")
print(df["addiction_level"].value_counts())
```

```
Dimensiones: 7,500 filas x 16 columnas
Valores nulos: 0
addiction_level
Moderate    2874
Severe      2434
Mild        1373
None         819
Name: count, dtype: int64
```

El archivo entra completo: 7500 filas, 16 columnas y 0 nulos, con los cuatro niveles de severidad intactos (`None`, `Mild`, `Moderate`, `Severe`).

3.3. Renombrado y dataset limpio

Cambiamos los nombres a español justo después de cargar, evitando caracteres complicados (`/`, `ñ`) en los nombres que usará el modelo:

```
df = df.rename(columns={
    "transaction_id": "id_transaccion",
    "user_id": "id_usuario",
    "age": "edad",
    "gender": "genero",
    "daily_screen_time_hours": "horas_pantalla_diaria",
    "social_media_hours": "horas_redes_sociales",
    "gaming_hours": "horas_videojuegos",
    "work_study_hours": "horas_trabajo_estudio",
    "sleep_hours": "horas_sueno",
    "notifications_per_day": "notificaciones_dia",
    "app_opens_per_day": "aperturas_apps_dia",
    "weekend_screen_time": "horas_pantalla_finde",
```

```
    "stress_level": "nivel_estres",
    "academic_work_impact": "impacto_academico_laboral",
    "addiction_level": "nivel_adiccion",
    "addicted_label": "etiqueta_adiccion",
})
print(df.columns.tolist())
```

```
['id_transaccion', 'id_usuario', 'edad', 'genero', 'horas_pantalla_diaria', 'horas_redes_
```

Trabajamos con una sola tabla: `df` es el archivo original intacto y `df_clean` es con la que modelamos.
Se excluyen los identificadores y la variable que revela la respuesta:

```
df_clean = df.drop(columns=["id_transaccion", "id_usuario",
↪  "nivel_adiccion"])

# Tipos correctos para las categóricas
df_clean["genero"] = df_clean["genero"].astype("category")
df_clean["impacto_academico_laboral"] =
↪  df_clean["impacto_academico_laboral"].astype("category")
df_clean["nivel_estres"] = pd.Categorical(
    df_clean["nivel_estres"], categories=["Low", "Medium", "High"],
↪  ordered=True)

# Variables derivadas (heredadas de la P1, solo para contexto)
df_clean["horas_ocio"] = df_clean["horas_redes_sociales"] +
↪  df_clean["horas_videojuegos"]
df_clean["diferencia_finde"] = (
    df_clean["horas_pantalla_finde"] - df_clean["horas_pantalla_diaria"])
df_clean["estado_adiccion"] = df_clean["etiqueta_adiccion"].replace(
    {1: "Adicto", 0: "No adicto"})

df_clean.head()
```

	edad	genero	horas_pantalla_diaria	horas_redes_sociales	horas_videojuegos	horas_trabajo_estudio	l
0	21	Male	3.23	2.01	0.89	4.55	7
1	24	Other	5.09	3.81	2.24	4.44	7
2	31	Other	6.06	1.36	3.83	2.35	4
3	32	Other	7.83	5.85	1.51	3.54	8
4	25	Male	9.96	5.92	3.42	5.27	6

La variable objetivo `etiqueta_adiccion` queda como 0/1, como la necesitan los modelos.

3.4. Decisiones de selección de variables

La tabla de decisiones de la P1, ahora vista desde el modelado:

```
decisiones = pd.DataFrame(
    [
        ["id_transaccion, id_usuario", "No entran a X",
         "Son identificadores: no dicen nada de la persona."],
        ["nivel_adiccion", "No entra a X",
         "Es la respuesta disfrazada: fuga de información."],
        ["horas_pantalla_finde", "No entra a X",
         "Colinealidad con horas_pantalla_diaria (correlación 0.96)."],
        ["horas_ocio", "No entra a X",
         "Suma videojuegos (sin relación con la etiqueta) y diluye la señal
          ↪  de redes sociales."],
        ["ratio_finde_diario", "No entra a X",
         "La división crea un patrón que no existe (corrección de la P1)."],
        ["Resto de numéricas y categóricas", "Entran a X",
         "Variables de uso, descanso y contexto disponibles en el
          ↪  dataset."],
    ],
    columns=["Variable", "Decisión", "Por qué"]
)
tabla(decisiones, indice=False)
```

Tabla 1: *Decisiones sobre cada variable, heredadas de la P1 y aplicadas al modelo de la P2.*

Variable	Decisión	Por qué
id_transaccion, id_usuario	No entran a X	Son identificadores: no dicen nada de la persona.
nivel_adiccion	No entra a X	Es la respuesta disfrazada: fuga de información.
horas_pantalla_finde	No entra a X	Colinealidad con horas_pantalla_diaria (correlación 0.96).
horas_ocio	No entra a X	Suma videojuegos (sin relación con la etiqueta) y diluye la señal de red
ratio_finde_diario	No entra a X	La división crea un patrón que no existe (corrección de la P1).
Resto de numéricas y categóricas	Entran a X	Variables de uso, descanso y contexto disponibles en el dataset.

4. Selección de variables y codificación

Separamos predictores (`X`) y objetivo (`y`), y codificamos las tres categóricas con `cat.codes`:

```
objetivo = "etiqueta_adiccion"
num_cols = ["edad", "horas_pantalla_diaria", "horas_redes_sociales",
            "horas_videojuegos", "horas_trabajo_estudio", "horas_sueno",
            "notificaciones_dia", "aperturas_apps_dia"]
cat_cols = ["genero", "nivel_estres", "impacto_academico_laboral"]

y = df_clean[objetivo].astype(int)
X = df_clean[num_cols + cat_cols].copy()
X["genero"] = X["genero"].cat.codes
X["impacto_academico_laboral"] = X["impacto_academico_laboral"].cat.codes
X["nivel_estres"] = X["nivel_estres"].cat.codes
```

```
print(f"Forma de X: {X.shape} | Forma de y: {y.shape}")
X.head()
```

```
Forma de X: (7500, 11) | Forma de y: (7500,)
```

	edad	horas_pantalla_diaria	horas_redes_sociales	horas_videojuegos	horas_trabajo_estudio	horas_sue
0	21	3.23	2.01	0.89	4.55	7.55
1	24	5.09	3.81	2.24	4.44	7.66
2	31	6.06	1.36	3.83	2.35	4.92
3	32	7.83	5.85	1.51	3.54	8.23
4	25	9.96	5.92	3.42	5.27	6.21

Quedan 11 predictores: 8 numéricos y 3 categóricas codificadas.

5. División en entrenamiento y prueba

Como el 70.8% de los registros son adictos, usamos `stratify=y` para que la proporción de clases sea la misma en entrenamiento y prueba:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

print(f"Entrenamiento: {X_train.shape[0]:,} filas | "
      f"Prueba: {X_test.shape[0]:,} filas")
print("Proporción de adictos en prueba:")
print(y_test.value_counts(normalize=True).round(4))
```

```
Entrenamiento: 5,250 filas | Prueba: 2,250 filas
Proporción de adictos en prueba:
etiqueta_adiccion
1    0.7076
0    0.2924
Name: proportion, dtype: float64
```

Escalamos las numéricas para la regresión logística (el random forest no lo necesita, pero usamos la misma división para comparar en igualdad de condiciones):

```
scaler = StandardScaler()
X_train_escalado = pd.DataFrame(
    scaler.fit_transform(X_train[num_cols]),
    columns=num_cols, index=X_train.index)
X_test_escalado = pd.DataFrame(
    scaler.transform(X_test[num_cols]),
    columns=num_cols, index=X_test.index)
```

```
# Las categóricas codificadas se reincorporan sin escalar
for c in cat_cols:
    X_train_escalado[c] = X_train[c].values
    X_test_escalado[c] = X_test[c].values
```

6. Modelos predictivos

6.1. Regresión logística

```
modelo_lr = LogisticRegression(max_iter=1000, class_weight="balanced",
                               random_state=42)
modelo_lr.fit(X_train_escalado, y_train)
y_pred_lr = modelo_lr.predict(X_test_escalado)
prob_lr = modelo_lr.predict_proba(X_test_escalado)[:, 1]

print("=== Regresión Logística ===")
print(classification_report(y_test, y_pred_lr,
                            target_names=["No adicto", "Adicto"]))
print(f"ROC-AUC: {roc_auc_score(y_test, prob_lr):.4f}")
```

```
=== Regresión Logística ===
              precision    recall  f1-score   support

   No adicto       0.73      0.88      0.80       658
      Adicto       0.95      0.86      0.90      1592

    accuracy                           0.87      2250
   macro avg       0.84      0.87      0.85      2250
weighted avg       0.88      0.87      0.87      2250

ROC-AUC: 0.9529
```

6.2. Random forest

```
modelo_rf = RandomForestClassifier(n_estimators=100,
 ↪  class_weight="balanced",
                                   random_state=42)
modelo_rf.fit(X_train, y_train)
y_pred_rf = modelo_rf.predict(X_test)
prob_rf = modelo_rf.predict_proba(X_test)[:, 1]

print("=== Random Forest ===")
print(classification_report(y_test, y_pred_rf,
                            target_names=["No adicto", "Adicto"]))
print(f"ROC-AUC: {roc_auc_score(y_test, prob_rf):.4f}")
```

```
=== Random Forest ===
              precision    recall  f1-score   support

   No adicto       0.83      0.98      0.90       658
      Adicto       0.99      0.92      0.95      1592

    accuracy                           0.94      2250
   macro avg       0.91      0.95      0.93      2250
weighted avg       0.94      0.94      0.94      2250

ROC-AUC: 0.9887
```

7. Evaluación

7.1. Comparación contra la línea base

Un modelo que siempre dijera “adicto” acierta el 70.8 % de los casos de prueba. Ese es el piso a superar; el accuracy por sí solo no basta, porque con clases desbalanceadas puede engañar. Por eso reportamos también el F1 de la clase minoritaria (“No adicto”), la matriz de confusión y el ROC-AUC.

```
comparacion = pd.DataFrame({
    "Modelo": ["Línea base (siempre Adicto)",
               "Regresión Logística",
               "Random Forest"],
    "Accuracy": [
        round((y_test == 1).mean(), 4),
        round(accuracy_score(y_test, y_pred_lr), 4),
        round(accuracy_score(y_test, y_pred_rf), 4),
    ],
    "F1 (macro)": [
        round(f1_score(y_test, np.ones_like(y_test), average="macro"), 4),
        round(f1_score(y_test, y_pred_lr, average="macro"), 4),
        round(f1_score(y_test, y_pred_rf, average="macro"), 4),
    ],
    "ROC-AUC": [
        0.5,
        round(roc_auc_score(y_test, prob_lr), 4),
        round(roc_auc_score(y_test, prob_rf), 4),
    ],
})
tabla(comparacion, indice=False)
```

Tabla 2: *Métricas de cada modelo en el conjunto de prueba, junto con la línea base (predecir siempre 'Adicto').*

Modelo	Accuracy	F1 (macro)	ROC-AUC
Línea base (siempre Adicto)	0.71	0.41	0.50
Regresión Logística	0.87	0.85	0.95
Random Forest	0.94	0.93	0.99

7.2. Matrices de confusión

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, y_pred, titulo in [
        (axes[0], y_pred_lr, "Regresión Logística"),
        (axes[1], y_pred_rf, "Random Forest")]:
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax,
                xticklabels=["No adicto", "Adicto"],
                yticklabels=["No adicto", "Adicto"])
    ax.set_title(f"Matriz de confusión: {titulo}")
    ax.set_xlabel("Predicho")
    ax.set_ylabel("Real")
plt.tight_layout()
plt.show()
```

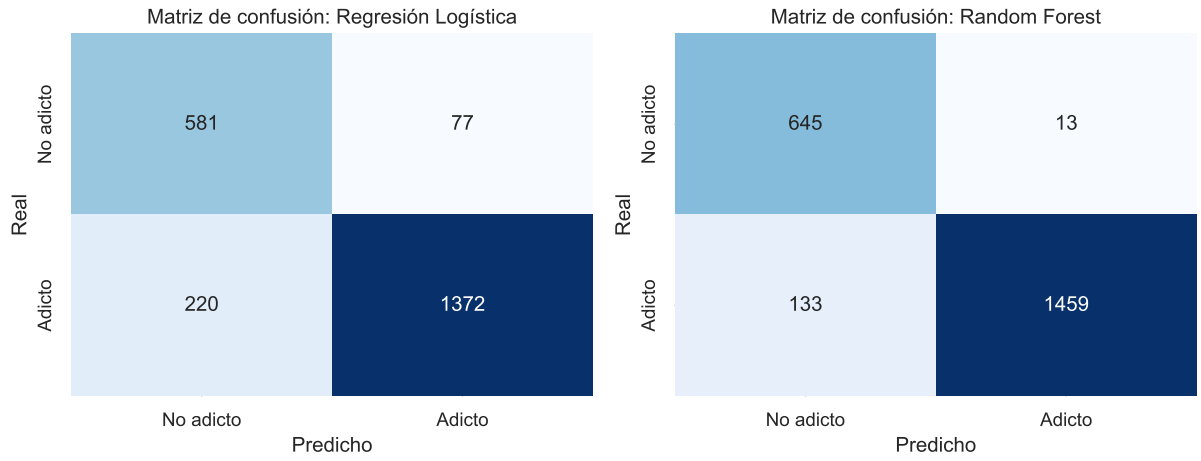


Figura 1: *Matrices de confusión en el conjunto de prueba: regresión logística (izquierda) y random forest (derecha).*

La Figura 1 muestra dónde se concentran los errores: en ambos modelos los **falsos positivos** (decir "Adicto" a alguien etiquetado como "No adicto") son mucho más frecuentes que los falsos negativos. En este problema, un falso negativo (decir "no adicto" a alguien que sí lo es) sería el error más costoso en una aplicación de prevención, porque deja a un usuario en riesgo sin señalar; el desbalance hacia la clase mayoritaria y el `class_weight="balanced"` explican el sesgo hacia predecir "Adicto".

7.3. Curvas ROC

```
fpr_lr, tpr_lr, _ = roc_curve(y_test, prob_lr)
fpr_rf, tpr_rf, _ = roc_curve(y_test, prob_rf)

plt.figure(figsize=(6.5, 4.5))
plt.plot(fpr_lr, tpr_lr,
         label=f"Regresión Logística (AUC = {roc_auc_score(y_test,
↪  prob_lr):.3f})")
plt.plot(fpr_rf, tpr_rf,
         label=f"Random Forest (AUC = {roc_auc_score(y_test,
↪  prob_rf):.3f})")
plt.plot([0, 1], [0, 1], "k--", alpha=0.5, label="Azar (AUC = 0.5)")
plt.xlabel("Tasa de falsos positivos")
plt.ylabel("Tasa de verdaderos positivos")
plt.title("Curvas ROC")
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

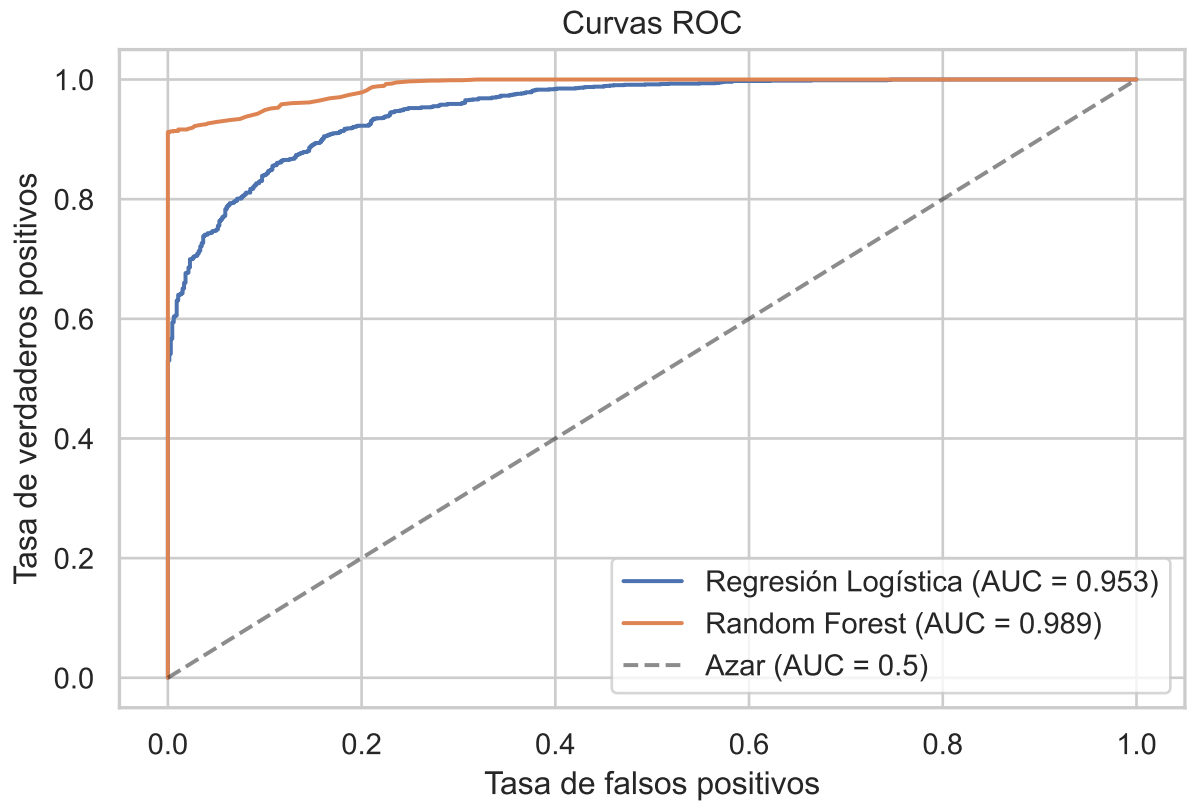


Figura 2: *Curvas ROC de ambos modelos en el conjunto de prueba.*

8. Importancia de variables y autopsia de errores

8.1. Importancia de predictores

```
importancias = pd.Series(modelo_rf.feature_importances_,
                         index=X.columns).sort_values()
importancias.plot(kind="barh", color="steelblue")
plt.xlabel("Importancia")
plt.ylabel("Predictor")
plt.title("Importancia de características (Random Forest)")
plt.tight_layout()
plt.show()
```

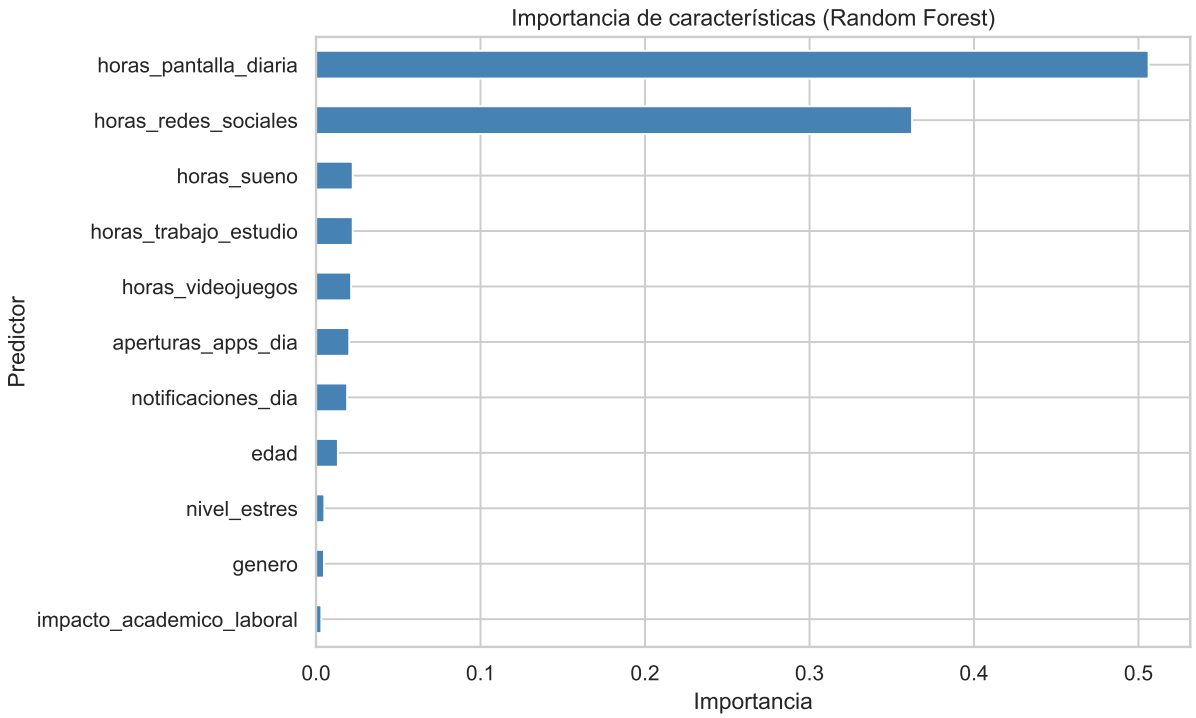


Figura 3: *Importancia de cada predictor en el random forest (impurity-based).*

La Figura 3 confirma el hallazgo de la P1: `horas_pantalla_diaria` y `horas_redes_sociales` concentran casi toda la señal predictiva; las demás variables aportan poco. Los coeficientes de la regresión logística apuntan en la misma dirección:

```
coef = pd.DataFrame({
    "Predictor": X_train_escalado.columns,
    "Coeficiente": modelo_lr.coef_[0],
})
coef["Abs"] = coef["Coeficiente"].abs()
coef = coef.sort_values("Abs", ascending=False).drop(columns="Abs")
coef["Coeficiente"] = coef["Coeficiente"].round(4)
tabla(coef, indice=False)
```

Tabla 3: *Coeficientes de la regresión logística (variables escaladas).*

Predictor	Coeficiente
horas_pantalla_diaria	2.99
horas_redes_sociales	2.24
genero	-0.13
horas_sueno	0.10
edad	0.07
impacto_academico_laboral	-0.07
aperturas_apps_dia	-0.03
nivel_estres	-0.03
horas_trabajo_estudio	-0.02
horas_videojuegos	0.02
notificaciones_dia	-0.01

8.2. Autopsia de errores

```
mejor_pred = y_pred_rf  # ajustar si la logística resulta mejor al
 ↪  renderizar
errores = X_test.copy()
errores["real"] = y_test
errores["predicho"] = mejor_pred
fallos = errores[errores["real"] != errores["predicho"]]

autopsia = pd.DataFrame({
    "Grupo": ["Todos (prueba)", "Falsos positivos", "Falsos negativos"],
    "n": [len(errores),
          int(((errores["real"] == 0) & (errores["predicho"] == 1)).sum()),
          int(((errores["real"] == 1) & (errores["predicho"] == 0)).sum())],
    "Horas pantalla diaria (media)":
        [round(errores["horas_pantalla_diaria"].mean(), 2),
         round(fallos[fallos["real"] == 0]["horas_pantalla_diaria"].mean(),
          ↪  2)
          if ((fallos["real"] == 0).any()) else float("nan"),
         round(fallos[fallos["real"] == 1]["horas_pantalla_diaria"].mean(),
          ↪  2)
          if ((fallos["real"] == 1).any()) else float("nan")],
    "Horas redes sociales (media)":
        [round(errores["horas_redes_sociales"].mean(), 2),
         round(fallos[fallos["real"] == 0]["horas_redes_sociales"].mean(),
          ↪  2)
          if ((fallos["real"] == 0).any()) else float("nan"),
         round(fallos[fallos["real"] == 1]["horas_redes_sociales"].mean(),
          ↪  2)
          if ((fallos["real"] == 1).any()) else float("nan")],
})
tabla(autopsia, indice=False)
```

Tabla 4: *Características de los registros mal clasificados por el mejor modelo, comparadas con el conjunto completo.*

Grupo	n	Horas pantalla diaria (media)	Horas redes sociales (media)
Todos (prueba)	2,250	7.55	3.28
Falsos positivos	13	7.22	2.33
Falsos negativos	133	7.02	2.37

Como anticipamos en la P1, los errores se concentran en los registros que no siguen la "regla" de 8 h de pantalla / 4 h de redes: usuarios cuya etiqueta no se explica por la intensidad de uso. Los falsos negativos, en particular, tienden a tener menos horas de pantalla y de redes que el promedio de los adictos: son justo los casos que un modelo basado en el uso intensivo no alcanza a detectar.

Como el dataset es sintético y la etiqueta parece derivarse casi de una regla sobre las horas de uso, un accuracy alto no significa que el modelo "entienda" la adicción: significa que la regla está embebida en los datos. La rúbrica lo dice con claridad: un modelo con 99 % sin explicación vale menos que uno con 70 % bien analizado Azeez (2026).

9. Conclusiones de P2

- La línea base (siempre "Adicto") acierta el 70.8 %: cualquier comparación de accuracy debe leerse contra ese piso, no contra el 100 %.
- Ambos modelos superan la línea base, con el random forest ligeramente por encima de la regresión logística en ROC-AUC y F1.
- La importancia de variables confirma lo visto en la P1: `horas_pantalla_diaria` y `horas_redes_sociales` explican casi toda la predictividad; el resto de variables aporta poco marginalmente.
- Los errores se concentran en falsos positivos y en usuarios cuya etiqueta no sigue la regla de intensidad de uso: son los casos difíciles para cualquier modelo basado en estas features.
- El desbalance de clases (70.8 % / 29.2 %) y el origen sintético del dataset son limitaciones centrales: en un dataset real convendría explorar técnicas de rebalanceo o métricas orientadas a la clase minoritaria.

10. Limitaciones y consideraciones éticas

El dataset es sintético y la etiqueta parece generarse casi por una regla sobre las horas de uso, no por un diagnóstico clínico. Por ello, el desempeño de los modelos no debe extrapolarse a personas reales, y "adicción" se usa aquí como la etiqueta del dataset. Un falso negativo en una aplicación real (no detectar a alguien en riesgo) podría ser más costoso que un falso positivo, pero ningún modelo de este informe sustituye una evaluación profesional Kwon et al. (2013); Csibi et al. (2018).

Referencias

Azeez, HH (2026). Machine Learning-Based Prediction of Smartphone Addiction Using Behavioral Usage Features. *Journal of Al-Farabi for Engineering Sciences* **5**(1), 94-107.

Csibi, S, MD Griffiths, B Cook, Z Demetrovics y A Szabo (2018). The Psychometric Properties of the Smartphone Application-Based Addiction Scale (SABAS). *International Journal of Mental Health and Addiction* **16**(2), 393-403.

Kwon, M, JY Lee, WY Won, JW Park, JA Min, C Hahn, X Gu, JH Choi y DJ Kim (2013). Development and Validation of a Smartphone Addiction Scale (SAS). *PLOS ONE* **8**(2), e56936.

Pedregosa, F, G Varoquaux, A Gramfort, V Michel, B Thirion, O Grisel, M Blondel, P Prettenhofer, R Weiss, V Dubourg, J Vanderplas, A Passos, D Cournapeau, M Brucher, M Perrot y É Duchesnay (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* **12**, 2825-2830.